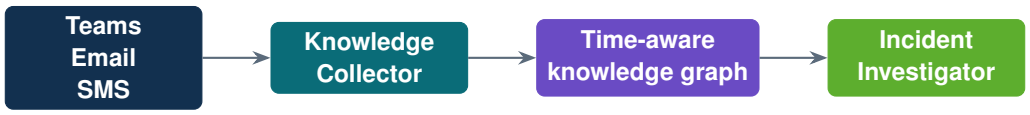
ELISA CHALLENGE • SINCE AI HACKATHON 2026 • TURKU

buildR

Capturing and Linking Human-Generated Contextual Metadata for Autonomous Network Operations

Technical documentation



Team	buildR
Challenge partner	Elisa Oyj
Event	Since AI Hackathon 2026, 6–8 November 2026, Turku
Repository	https://github.com/shahriarromy/sinceaihackathon2026-elisa
Version	1.0 – November 7, 2026

Contents

1 Summary **3**

2 The problem **4**

2.1 Elisa's challenge . . . 4

2.2 Two real examples from Elisa . . . 4

2.3 Requirements we derived . . . 4

3 Solution overview **5**

3.1 Idea in three steps . . . 5

3.2 Following Elisa's reference architecture . . . 5

3.3 What makes it different . . . 6

4 System architecture **7**

4.1 Component view . . . 7

4.2 Containers and ports . . . 7

4.3 Start-up order and health checks . . . 7

4.4 Technology stack . . . 8

5 Data model (Neo4j knowledge graph) **8**

5.1 Design principles . . . 8

5.2 Entity-relationship diagram . . . 9

5.3 Node labels . . . 11

5.4 The `ContextAssertion` node . . . 11

5.5 Time validity in practice . . . 12

5.6 Key Cypher queries . . . 12

5.7 Vector store (Chroma) . . . 13

6 AI agents **14**

6.1 Agent 2 – Knowledge Collector . . . 14

6.2 Agent 1 – Incident Investigator . . . 15

7 Services, tools and API **16**

7.1 MCP servers and their tools . . . 16

7.2 REST API (FastAPI, port 8000) . . . 16

7.3 End-to-end sequence . . . 17

7.4 Configuration . . . 18

8 User interface **19**

9 Installation and running **21**
9.1 Prerequisites . . . 21
9.2 Installation flow . . . 21
9.3 Step by step . . . 21
9.4 Automated acceptance demo . . . 22
9.5 Reset to a clean demo . . . 22
9.6 Running tests without Docker . . . 22
9.7 Troubleshooting . . . 22

10 Demo walkthrough **23**
10.1 Step 1 – Investigate: no prior fix found . . . 23
10.2 Step 2 – One click: capture team knowledge . . . 23
10.3 Step 3 – Investigate again: the answer with its source . . . 24
10.4 Step 4 – Time and conflicts . . . 25

11 Synthetic data: what we generated and how **26**
11.1 Principle: facts first, then words . . . 26
11.2 The synthetic network . . . 26
11.3 Communication channels . . . 26
11.4 Seeded context assertions . . . 27
11.5 Five reference work orders . . . 27

12 Testing and validation **27**
12.1 Test suite . . . 28
12.2 Acceptance test (`scripts/demo.sh`) . . . 28
12.3 What correctness means here . . . 28

13 How buildR meets Elisa's evaluation criteria **29**

14 Limitations and roadmap **30**
14.1 Current limitations (honest status of the prototype) . . . 30
14.2 Roadmap . . . 30
14.3 Security and privacy . . . 30

A Repository structure **31**

B Example source data **32**

C Glossary **33**

1 Summary

Elisa's network can already find most faults with AI. But the autonomous cycle still stops when the *next step* depends on knowledge that only lives in people's chats, emails and memory. Elisa calls this **silent metadata**.

buildR is a working prototype that:

1. **Listens** to the channels engineers already use (Teams, email, SMS) through small MCP connector services. Specialists do not fill in any form.
2. **Links** every useful tip to the right network entity (device, model, site, access point, work order) in a **Neo4j knowledge graph**, as a `ContextAssertion` with `valid_from` / `valid_to`, source and author. The same text is indexed in a **Chroma** vector store for semantic search.
3. **Helps**: when a new incident opens, an AI agent reads the graph and the vector store, keeps only advice that is valid *now*, flags conflicting advice, and returns an ordered action plan with its source.

Result on Elisa's own example

Open incident `WO-2026-1107-003` (TRN-117, TX-2000, firmware 4.1): SFP replaced, interface UP, alarms clear, traffic zero. **Before capture** the Investigator finds no fix (confidence 40 %). After **one click** the Collector reads the team channels and stores the expert workaround. **After capture** the Investigator returns: *check counters* → *disable interface* → *wait 30 s* → *enable; do not reboot or swap the SFP*, with source, author and validity (confidence 100 %).

Table 1: The prototype at a glance

Item	Value
Runtime	10 Docker containers, one `docker compose up -d`; works offline except optional LLM calls
Agents	2 LangGraph agents: Knowledge Collector (write) and Incident Investigator (read-only)
Protocol	Model Context Protocol (FastMCP, HTTP/SSE) for every tool the agents use
Storage	Neo4j 5.26 (time-aware graph) + ChromaDB (MiniLM embeddings, pre-downloaded)
API	FastAPI on port 8000, OpenAPI docs at `/docs`, SSE streaming of agent steps
User interface	React + Mantine web app on port 3000
Demo data	Synthetic: 14 sites, 5 regions, 6 Teams threads, 5 emails, 1 SMS thread, 11 seeded assertions

2 The problem

2.1 Elisa's challenge

Elisa wants to build the world's first and most advanced autonomous network. AI already supports root-cause analysis and fault correction. One of the biggest remaining barriers is **silent metadata**: operational knowledge created in calls, emails and Teams chats that is not stored in any central system. When it is missing, the autonomous workflow stops and a human has to step in.

Guiding question: *How could we collect silent metadata without creating additional work for our specialists?*

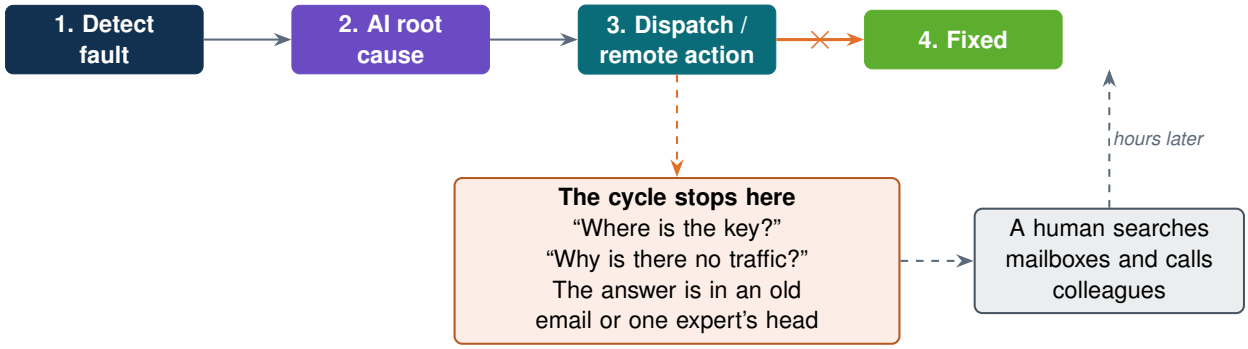


Figure 1: Where the autonomous cycle breaks today.

2.2 Two real examples from Elisa

Table 2: The two cases in the challenge brief and how they appear in our demo data

	Access information	Expert troubleshooting
What happened	AI found a faulty device in Northern Finland. The engineer was stopped by a locked barrier on a private road.	After an SFP swap the interface is UP, optics normal, alarms clear, but traffic does not recover.
Where the answer was	Only in an email (where the key is).	Only in Erkki's memory, a Teams chat and an email: disable and re-enable the interface.
Demo work order	`WO-2026-0114-002`, TRN-233, site SAL-03 Salla	`WO-2026-0312-007`, TRN-042 (captured) and the open `WO-2026-1107-003`, TRN-117
Time lost	1 h 45 min waiting at the barrier	19 min "UP but no traffic", risk of a second SFP or a reboot

2.3 Requirements we derived

- **R1 Zero extra work:** capture from Teams, email and SMS automatically; no forms for experts.
- **R2 Correct linking:** bind each piece of knowledge to the right entity (device, model, site, access point, work order).
- **R3 Time validity:** every assertion has `valid_from` / `valid_to`; outdated advice is closed, not deleted.
- **R4 Trust:** keep the source, author and confidence; show conflicting advice instead of hiding it.
- **R5 Agent-ready:** an AI agent must retrieve the knowledge and propose the next step *before* hardware is replaced.
- **R6 No fabrication:** if nothing is known, the agent must say so.
- **R7 Safety:** never store secrets (gate codes); store who holds them.

3 Solution overview

3.1 Idea in three steps

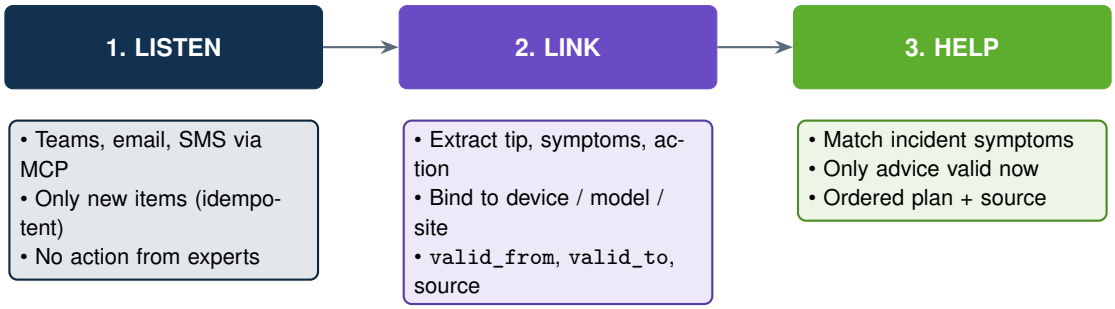


Figure 2: The three steps of buildR.

3.2 Following Elisa's reference architecture

Elisa describes the network in time-aware layers. Each *context assertion* is bound to a graph entity and carries `valid_from` / `valid_to`, so all layers can be cross-referenced at any point in time (Figure 3). We implemented this model directly in Neo4j (Figure 4).

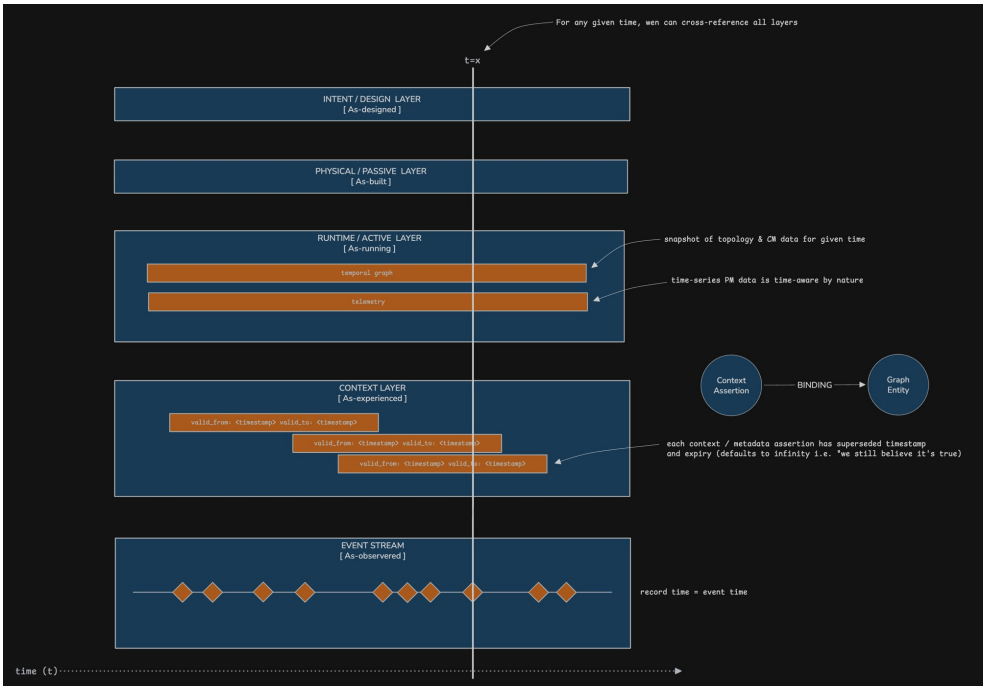


Figure 3: Elisa's data-layer and context architecture (from the challenge brief).

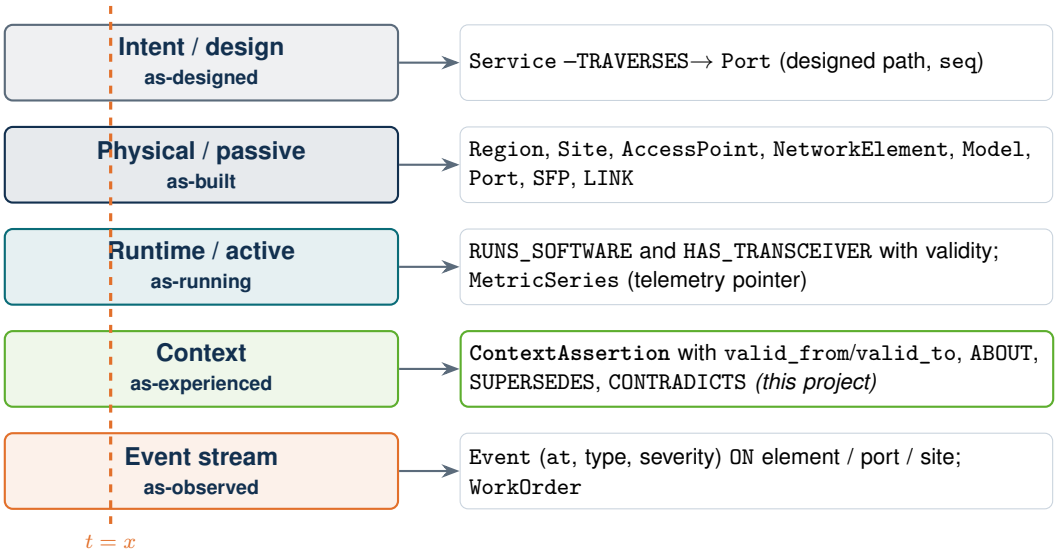


Figure 4: How each of Elisa's layers maps to labels in our Neo4j graph. A vertical cut at time t selects what was true at t in every layer.

3.3 What makes it different

- **Graph + vector search together.** Plain search returns *both* the old and the new answer. The graph knows which one is still valid (`valid_to`, `SUPERSEDES`), which device it applies to, and who said it.
- **Conflicts are kept, not resolved silently.** Two experts can disagree (`CONTRADICTS`); the agent shows both and recommends the least risky step first.
- **MCP everywhere.** Every data source is a small MCP server. Replacing a dummy channel with a real Microsoft Graph or SMS gateway connector does not change the agents.
- **Read/write separation.** The Investigator can only read (`neo4j-mcp-ro` blocks write keywords); only the Collector writes.

4 System architecture

4.1 Component view

Figure 5 shows all ten containers, grouped by layer. Arrows show the direction of calls. Only the web app (3000), the agents API (8000) and the Neo4j browser (7474/7687) are published to the host; everything else is internal to the Docker network `ops_net`.

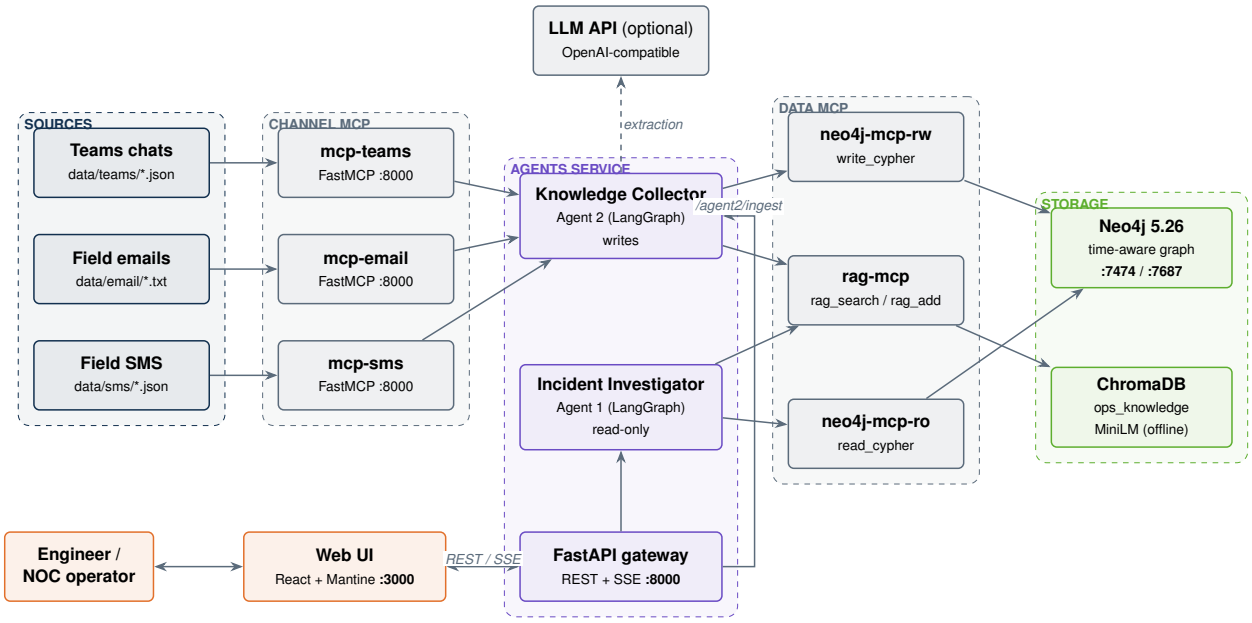


Figure 5: Detailed component architecture (all containers of `docker-compose.yml`).

4.2 Containers and ports

Table 3: Docker Compose services

Service	Container	Host port	Role / technology
`frontend`	ops-frontend	**3000**	React 18, Vite, TypeScript, Mantine v7; nginx serves the build and proxies `/api/` to the agents service
`agents`	ops-agents	**8000**	FastAPI gateway + both LangGraph agents, Python 3.12 (uv)
`neo4j`	ops-neo4j	**7474** / **7687**	Neo4j 5.26 Community with APOC; volume `neo4j_data`
`neo4j-mcp-ro`	ops-neo4j-mcp-ro	internal	FastMCP Cypher adapter, `READ_ONLY=true` (write keywords rejected)
`neo4j-mcp-rw`	ops-neo4j-mcp-rw	internal	FastMCP Cypher adapter with `write_cypher`
`chromadb`	ops-chromadb	internal	Chroma vector store; volume `chroma_data`
`rag-mcp`	ops-rag-mcp	internal	FastMCP wrapper: `rag_search`, `rag_add`, `rag_list_sources`
`mcp-teams`	ops-mcp-teams	internal	FastMCP, reads `./data/teams` (read-only mount)
`mcp-email`	ops-mcp-email	internal	FastMCP, reads `./data/email` (read-only mount)
`mcp-sms`	ops-mcp-sms	internal	FastMCP, reads `./data/sms` (read-only mount)

4.3 Start-up order and health checks

Every service has a health check, and `depends_on: condition: service_healthy` enforces the order in Figure 6. The agents container only accepts traffic when Neo4j, both Neo4j MCPs, the RAG MCP and all three channel MCPs are healthy.

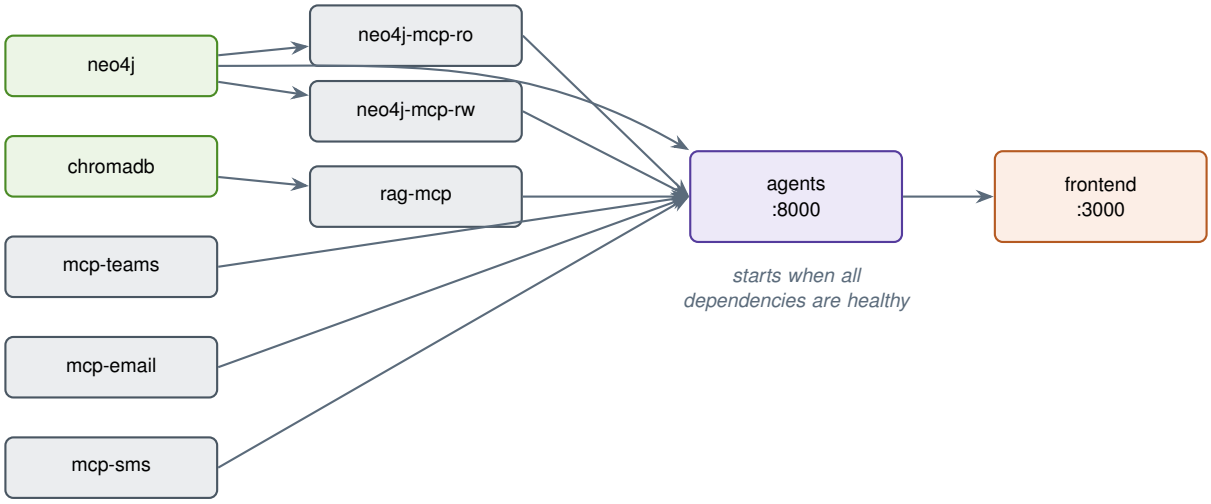


Figure 6: Container start-up dependencies (`depends_on: service_healthy`).

4.4 Technology stack

Table 4: Main technologies

Area	Technology	Why
Agents	LangGraph	Explicit state graphs: every step is visible and streamed to the UI
Tool protocol	FastMCP, `langchain-mcp-adapters`	Standard way for agents to use tools; each agent gets only its own tools
LLM	Featherless AI, called via `langchain-openai` (`ChatOpenAI`)	Serverless open-weight models behind an OpenAI-compatible endpoint; temperature 0, structured output
Graph	Neo4j 5.26	Recommended by Elisa; native relationships and temporal properties
Vector search	ChromaDB + all-MiniLM-L6-v2 (ONNX)	Semantic symptom search; model baked into the image (offline)
API	FastAPI, Pydantic v2, sse-starlette	Typed schemas, OpenAPI docs, Server-Sent Events
Frontend	React 18, Vite 6, TypeScript, Mantine 7	Fast to build, clean light/dark UI
Packaging	uv, Docker Compose	Reproducible Python environments, one-command start

5 Data model (Neo4j knowledge graph)

5.1 Design principles

- **Readable business keys.** Every node has a unique string `id` (e.g. `TRN-117`, `SAL-03`, `WO-2026-1107-003`). The same id appears in chats, tickets and the graph, which makes linking checkable.
- **Time on relationships too.** Things that change in the network (software running, SFP in a port, fibre link) carry `valid_from` / `valid_to` on the relationship, so we can ask “which firmware did TRN-117 run on 7 Nov?”.
- **Context is a first-class node.** Human knowledge is a `ContextAssertion` node bound with `ABOUT`

to the entity it describes. It is never a free-text field on a device.

- **No text in the graph that belongs to the source.** Conversations stay in their channel; the assertion keeps a `source_ref` (e.g. `teams/TH-T-001.json#seq=4`). The full text is indexed in Chroma.

5.2 Entity-relationship diagram

Figure 7 is the complete schema as seeded by `data/seed/knowledge_graph_seed.cypher`. Blue = physical network (as-built), teal = runtime and operations, orange = events and access, green = context layer.

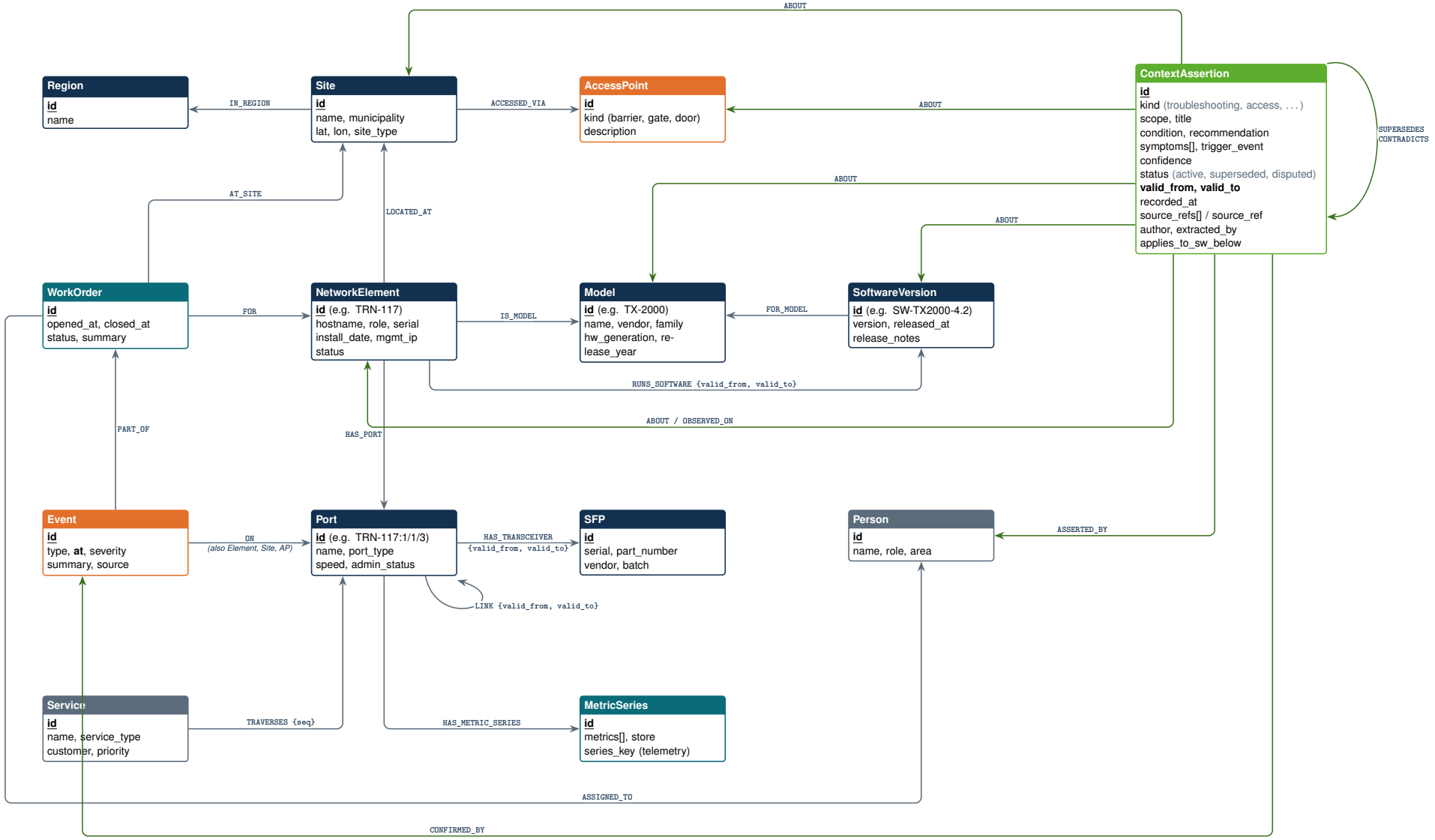


Figure 7: Entity-relationship diagram of the Neo4j property graph. **id** = unique key (uniqueness constraint). Relationship properties in braces. Green arrows belong to the context layer.

5.3 Node labels

Table 5: Node labels, number of seeded nodes and meaning

Label	Seeded	Meaning and key properties
`Region`	5	Finnish region (Uusimaa, Varsinais-Suomi, Pirkanmaa, Pohjois-Pohjanmaa, Lappi)
`Site`	14	Physical location: hub, rooftop, street cabinet, rural mast; coordinates, municipality
`AccessPoint`	4	How a site is reached: barrier, gate, badge door, track (e.g. `AP-SAL-03-BARRIER`)
`Model`	4	Hardware model: TX-2000 (legacy, 2017), TX-3000, MW-500, AG-100
`SoftwareVersion`	6	Firmware per model, e.g. `SW-TX2000-4.2` "fixes forwarding table re-initialisation after optical hot-swap"
`NetworkElement`	23	Device: transmission node, aggregation router, microwave radio
`Port`	94	Interface on an element; id = `<element>:<port>`
`SFP`	88	Optical module with serial, part number and batch
`Service`	6	Customer or internal service with its designed path (`TRAVERSES` with `seq`)
`Person`	7	Expert, field engineer or NOC operator (display name and role only)
`WorkOrder`	5	Ticket / incident: opened, closed, status, summary
`Event`	104	Alarms, RCA results, dispatches, SFP swaps, interface state changes, access events
`ContextAssertion`	11	Captured human knowledge (seed) + new ones created by the Collector
`MetricSeries`	4	Pointer to telemetry (oper status, RX power, packet and CRC counters)

5.4 The `ContextAssertion` node

This is the heart of the solution: one node per piece of reusable knowledge.

Table 6: `ContextAssertion` properties

Property	Type	Meaning
`id`	string	`CA-0001...` in the seed; `CA-<sha256(title:source)[:8]>` when created by the Collector (deterministic ⇒ idempotent)
`kind`	string	`troubleshooting`, `access_information`, `known_defect`, `other`
`scope`	string	`model`, `element`, `site` (how wide the knowledge applies)
`title`, `text`	string	Short title and full explanation
`condition`, `symptoms[]`	string, list	When the advice applies, e.g. `oper_status=UP`, `rx_packets_delta=0`
`trigger_event`	string	Context trigger, e.g. `SFP_REPLACED`
`recommendation`	string	Ordered next steps, including what *not* to do
`confidence`	float	0–1; lowered for disputed advice
`status`	string	`active`, `superseded`, `disputed`
`valid_from`	datetime	When the knowledge started to be true (time of the first message stating it)
`valid_to`	datetime / null	**null = still believed true**; set when newer knowledge supersedes it
`recorded_at`	datetime	When the system learned it (system time vs. world time)
`source_ref(s)`	string / list	Pointer to the evidence, e.g. `teams/TH-T-001.json#seq=8`
`author`, `extracted_by`	string	Who said it, and which process extracted it
`applies_to_sw_below`	string	Optional version bound, e.g. advice valid only below firmware 4.2

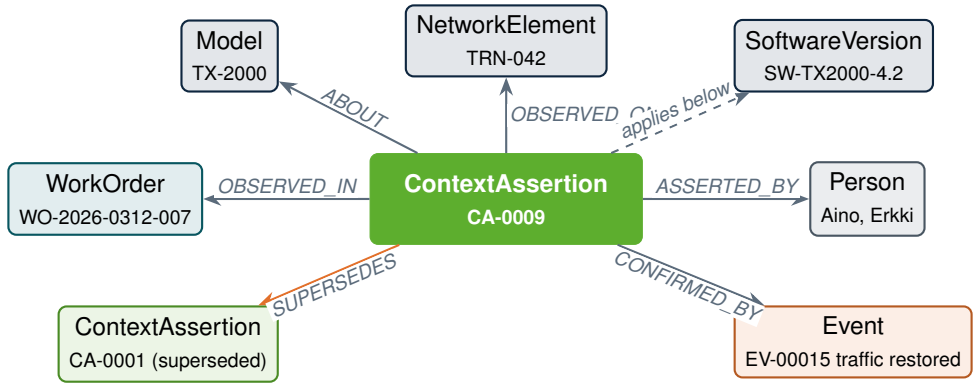


Figure 8: One assertion and its links: what it is about, where it was seen, who said it, which event proved it, and which older advice it replaces.

5.5 Time validity in practice

Each assertion lives on a timeline. Figure 9 shows the two seeded supersession chains.

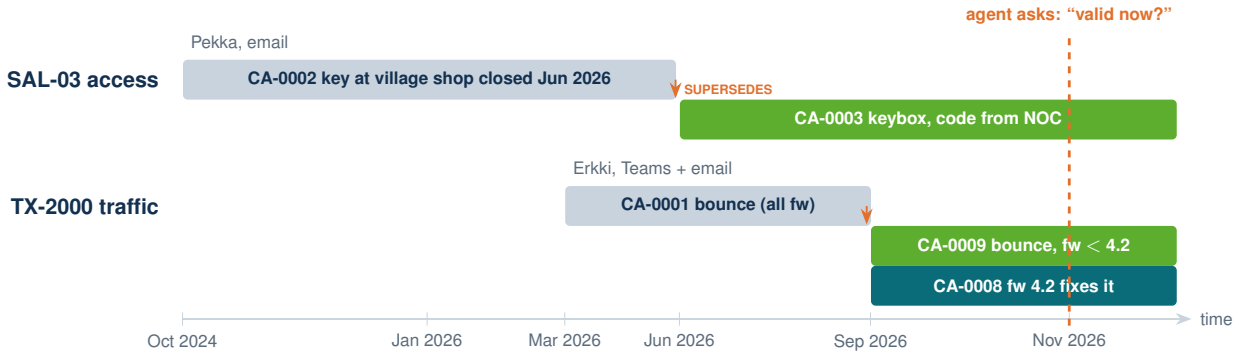


Figure 9: Validity windows. Grey bars are closed (`valid_to` set); green bars are still believed true. At query time the agent keeps only bars that cross the dashed line and are not the target of `SUPERSEDES`.

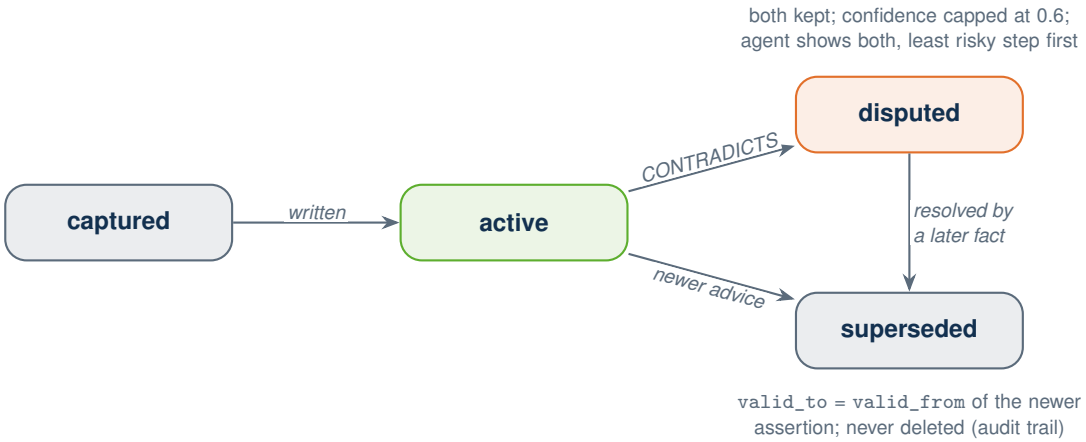


Figure 10: Lifecycle of a context assertion.

5.6 Key Cypher queries

Retrieval by the Investigator (`agent1/graph.py`): only assertions linked to the incident's element, model, site, work order or device class, valid now, and not superseded.

```
MATCH (ca:ContextAssertion)-[:ABOUT|OBSERVED_IN]->(target)
WHERE (target.id IN [$element_id, $model_id, $site_id, $incident_id]
       OR target.name = $device_class)
  AND (ca.valid_to IS NULL OR ca.valid_to > datetime())
  AND NOT ()-[:SUPERSEDES]->(ca)
OPTIONAL MATCH (ca)-[:SUPERSEDES]->(old_ca:ContextAssertion)
RETURN ca.id, ca.title, ca.recommendation, ca.confidence, ca.source_ref,
       toString(ca.valid_from) AS valid_from, toString(ca.valid_to) AS valid_to,
       collect(DISTINCT old_ca.id) AS supersedes
ORDER BY ca.confidence DESC
```

Write by the Collector (`agent2/graph.py`): idempotent `MERGE` on a deterministic id.

```
MERGE (ca:ContextAssertion {id: $id})
SET ca.title = $title, ca.text = $text, ca.kind = $kind,
    ca.recommendation = $recommendation, ca.confidence = $confidence,
    ca.valid_from = datetime($valid_from),
    ca.valid_to = CASE WHEN $valid_to IS NOT NULL THEN datetime($valid_to) ELSE null END,
    ca.source_ref = $source_ref, ca.author = $author, ca.recorded_at = datetime()
// then, per resolved mention:
MATCH (ca:ContextAssertion {id: $ca_id}), (ne:NetworkElement {id: $ent})
MERGE (ca)-[:ABOUT]->(ne)
```

Supersession (old advice closed, not deleted):

```
MATCH (new_ca:ContextAssertion {id: $new_id}), (old_ca:ContextAssertion {id: $old_id})
MERGE (new_ca)-[:SUPERSEDES]->(old_ca)
SET old_ca.valid_to = new_ca.valid_from
```

Constraints (excerpt of the seed):

```
CREATE CONSTRAINT contextassertion_id IF NOT EXISTS
  FOR (n:ContextAssertion) REQUIRE n.id IS UNIQUE;
CREATE CONSTRAINT networkelement_id IF NOT EXISTS
  FOR (n:NetworkElement) REQUIRE n.id IS UNIQUE;
CREATE INDEX event_at IF NOT EXISTS FOR (n:Event) ON (n.at);
CREATE INDEX assertion_status IF NOT EXISTS FOR (n:ContextAssertion) ON (n.status);
```

5.7 Vector store (Chroma)

The RAG store complements the graph with semantic search over the original text.

Table 7: Chroma collection `ops_knowledge`

Field	Content
Document	One chunk per logical message group (a whole Teams thread, one email, one SMS thread)
Embedding	Chroma default `all-MiniLM-L6-v2` (ONNX), pre-downloaded in the image build ⇒ offline
Id	Deterministic hash of `source_ref` ⇒ `rag_add` is idempotent
Metadata	`source_type`, `source_ref`, `author`, `timestamp`, `entity_ids` (comma-separated, Chroma metadata must be scalar), `incident_id`, `kind`

6 AI agents

Both agents live in the `agents` service and are built as **explicit LangGraph state graphs**, not as a black-box tool loop. Every node appends a human-readable line to `step_events`; the API streams these lines to the UI, so the user sees what the agent is doing (Figure 18 in Section 8).

Table 8: The two agents

	Agent 2 – Knowledge Collector	Agent 1 – Incident Investigator
Purpose	Turn conversations into context assertions	Turn an incident into an ordered action plan
Trigger	`POST /agent2/ingest` (button, schedule or webhook)	`POST /agent1/report[/stream]`
Tools (MCP)	mcp-teams, mcp-email, mcp-sms, neo4j-mcp-rw, rag-mcp	neo4j-mcp-ro, rag-mcp
Writes?	Yes (graph + vector store)	**Never** (read-only MCP)
LLM use	Structured extraction (`with_structured_output`)	Optional; report is assembled from graph facts
Output	`Agent2IngestResponse`: assertions, incidents touched, RAG ids	`Agent1ReportResponse` (Table 9)

6.1 Agent 2 – Knowledge Collector

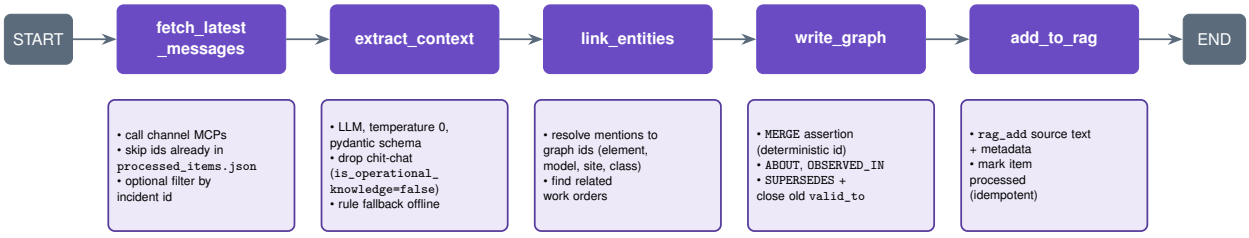


Figure 11: LangGraph of the Knowledge Collector (`agent2/graph.py`).

Extraction schema. The LLM (an open-weight model served through Featherless AI) must fill this pydantic model. If the message has no reusable knowledge, it returns `is_operational_knowledge = false` and nothing is stored (precision before recall).

```python
class ExtractedAssertion(BaseModel):
    is_operational_knowledge: bool # False for chit-chat
    title: str # short title
    text: str # full, concise explanation
    kind: str = "troubleshooting_knowledge" # | access_information | other
    entity_mentions: List[str] # e.g. ["TRN-042", "TX-2000", "SAL-03"]
    conditions: List[str] # e.g. ["oper_status=UP", "rx_packets_delta=0"]
    recommended_action: str # ordered steps, incl. what NOT to do
    confidence: float = 0.9
    valid_from: Optional[str] # ISO timestamp of the message
    valid_to: Optional[str] # only if the text says it is outdated
    source_ref: str # e.g. "teams/TH-T-001.json"
    author: str # display name only (no other personal data)
```

Idempotency. Running the ingest twice changes nothing: channel items are tracked in `processed_items.json`; assertion ids are `CA-` + the first 8 hex digits of `sha256(title:source_ref)`; graph writes use `MERGE`; RAG ids are a hash of `source_ref`.

Offline fallback. When no `LLM_API_KEY` is set, or the LLM call fails, a deterministic rule-based extractor handles the demo scenarios. This keeps the demo reliable without internet.

6.2 Agent 1 – Incident Investigator

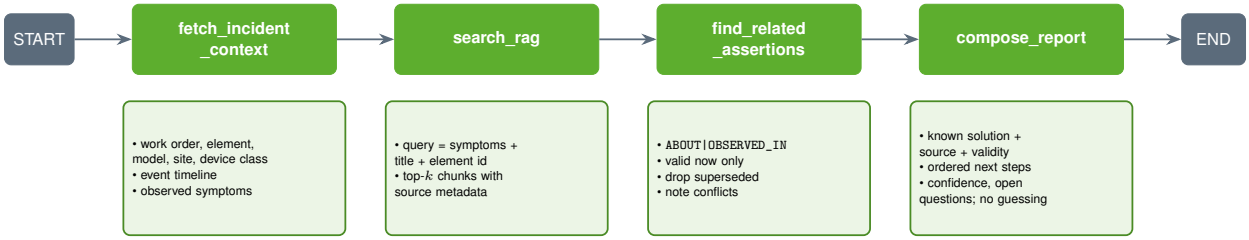


Figure 12: LangGraph of the Incident Investigator (`agent1/graph.py`).

Decision logic in `compose_report`

The Investigator prefers facts from the graph, uses the vector store only as a second source, and never invents a fix.

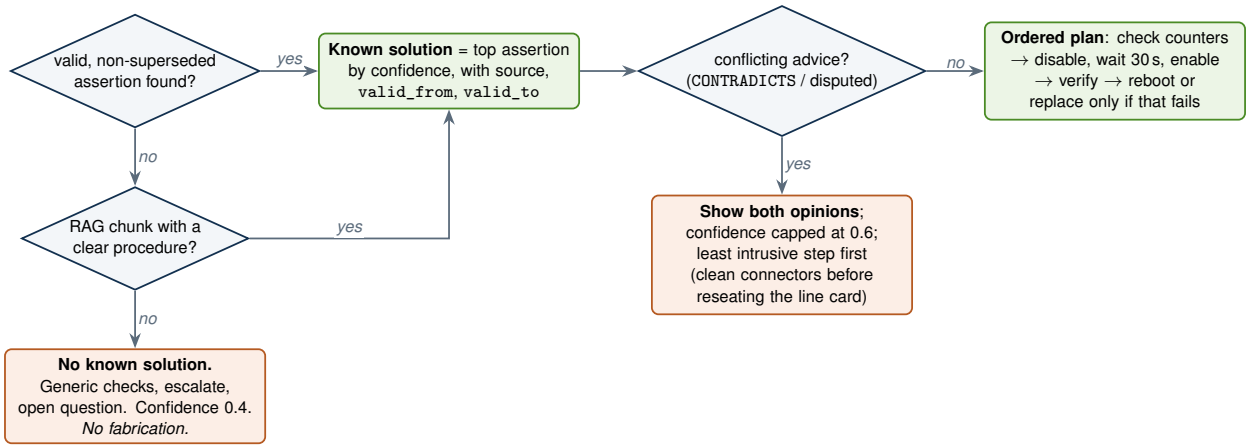


Figure 13: How the Investigator builds its answer.

Table 9: Report returned by the Investigator (`Agent1ReportResponse`)

Field	Type	Content
`summary`	string	Incident id and title
`timeline`	list	Events in time order (fault, dispatch, SFP replaced, interface UP, . . .)
`symptoms`	list	Observed symptom codes
`problem_description`	string	Plain-language description; says if a known workaround exists
`known_solution`	object / null	`title`, `recommendation`, `source_ref`, `confidence`, `valid_from`, `valid_to`; **null when nothing is known**
`suggested_next_steps`	list	Ordered steps, cheapest and safest first
`confidence`	float	0–1
`open_questions`	list	What is missing, or which experts disagree

7 Services, tools and API

7.1 MCP servers and their tools

All tool servers use **FastMCP** over HTTP/SSE (`http://<service>:8000/sse`) and expose `/health`. Channel servers are stateless and read their data from read-only volume mounts; the agents service owns the "already processed" state.

Table 10: MCP tools

Server	Tool	What it does	Used by
`mcp-teams`	`list_conversations(since?)`	Teams threads, optionally after a timestamp	Collector
	`get_conversation(id)`	One full thread with all messages	Collector
	`get_latest_conversations(n)`	The newest n threads	Collector
`mcp-email`	`list_emails(since?)`	Same for email files	Collector
	`get_email(id)`		
	`get_latest_emails(n)`		
`mcp-sms`	`list_messages(since?)`	SMS threads	Collector
	`get_latest_messages(n)`		
`rag-mcp`	`rag_search(query, entity_id?, k=5)`	Semantic search with optional entity filter; returns text, metadata, distance	both
	`rag_add(text, metadata)`	Idempotent insert (id = hash of `source_ref`)	Collector
	`rag_list_sources()`	Indexed sources and counts (debug)	–
`neo4j-mcp-ro`	`get_schema()`	Read queries only; rejects `CREATE`, `MERGE`, `DELETE`, `SET`, `DROP`, `REMOVE`	Investigator
	`read_cypher(query, params)`		
`neo4j-mcp-rw`	`get_schema()`, `read_cypher`	Writes; returns counters (nodes and relationships created)	Collector
	`write_cypher(query, params)`		

All channel items are normalised to the same JSON shape before extraction:

```
{"id": "TH-T-006", "source_type": "teams", "source_ref": "teams/TH-T-006.json",
 "timestamp": "2026-11-07T08:48:00+02:00", "author": "Mikko Virtanen",
 "participants": ["Jari Järvinen", "Mikko Virtanen", "Sanna Saarinen"],
 "subject": "TRN-117 resolution - SFP traffic recovered", "body": "..."}
```

7.2 REST API (FastAPI, port 8000)

Interactive OpenAPI documentation is served at `http://localhost:8000/docs`.

Table 11: REST endpoints

Method	Path	Description
GET	`/health`	Status of Neo4j and every MCP endpoint
POST	`/agent1/report`	Run the Investigator. Body `{"incident_id": "WO-2026-1107-003"}` or `{"query": "..."}`
POST	`/agent1/report/stream`	Same, as Server-Sent Events: one `step` event per graph node, then one `report` event
POST	`/agent2/ingest`	Run the Collector. Body `{"sources": ["teams","email","sms"], "limit": null, "incident_id": "..."}`
GET	`/incidents`	All work orders with status, element, site and event timeline
GET	`/incidents/{id}`	One work order with details
POST	`/seed-graph`	Load `knowledge_graph_seed.cypher` (idempotent)
POST	`/rag/search`	Pass-through to `rag_search` for debugging

Example: capture, then investigate.

```
# 1. Capture knowledge from all channels for the open incident
curl -s -X POST http://localhost:8000/agent2/ingest \
     -H "Content-Type: application/json" \
     -d '{"sources":["teams","email","sms"],"incident_id":"WO-2026-1107-003"}'

# 2. Ask the Investigator
curl -s -X POST http://localhost:8000/agent1/report \
     -H "Content-Type: application/json" \
     -d '{"incident_id":"WO-2026-1107-003"}'
```

Example SSE stream (`/agent1/report/stream`):

```
event: step
data: {"node": "fetch_incident_context", "message": "Retrieved element context and historical
    events"}
event: step
data: {"node": "search_rag", "message": "Queried Chroma vector database for matching symptoms"}
event: step
data: {"node": "find_related_assertions", "message": "Located active, valid ContextAssertions"}
event: report
data: {"summary": "Incident Report for WO-2026-1107-003 ...", "confidence": 1.0, ...}
```

7.3 End-to-end sequence

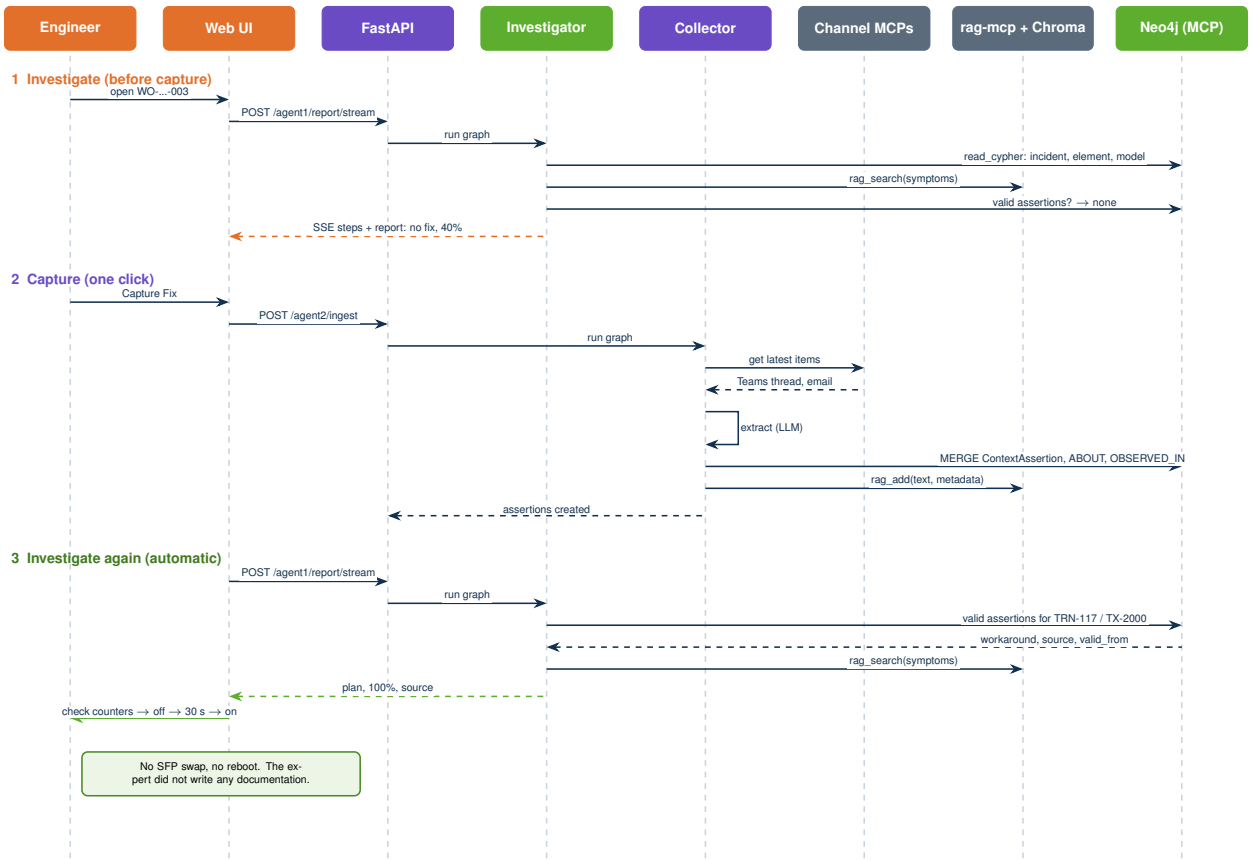


Figure 14: End-to-end sequence of the demo: investigate → capture → investigate again.

7.4 Configuration

All configuration is done with environment variables (`.env`, template in `.env.example`). No secrets are in code.

Table 12: Environment variables

Variable	Default	Purpose
NEO4J_URI	bolt://neo4j:7687	Graph database connection
NEO4J_USER / NEO4J_PASSWORD	neo4j / (set in .env)	Graph credentials (change for any shared deployment)
LLM_BASE_URL	https://api.openai.com/v1	Any OpenAI-compatible endpoint; in our deployment, Featherless AI (https://api.featherless.ai/v1)
LLM_API_KEY	empty	If empty, the deterministic offline extractor is used
LLM_MODEL	gpt-4o-mini	Model for structured extraction
CHROMA_HOST / CHROMA_PORT	chromadb / 8000	Vector store
COLLECTION_NAME	ops_knowledge	Chroma collection
*_MCP_URL	http://<svc>:8000/sse	Six MCP endpoints (teams, email, sms, rag, neo4j ro/rw)
READ_ONLY	true / false	Mode of each Neo4j MCP container
STATE_STORAGE_PATH	/app/processed_items.json	Collector's processed-items file

8 User interface

The web app (`http://localhost:3000`) is built with React 18, Vite, TypeScript and Mantine UI v7. nginx serves the production build and proxies `/api/` to the agents service. It has three views.

Table 13: Views of the web app

View	Component	What the user can do
Incidents & Investigation	`IncidentsView.tsx`	See open and resolved work orders, run the Investigator with live step trace, capture team knowledge with one click
Resolution Timelines	`SequencesFlowView.tsx`	See five work orders as swimlanes and how knowledge was missed, captured, disputed, confirmed and applied
Knowledge Collector	`SettingsView.tsx`	Seed the graph, run capture, check service health, read the raw Teams / email / SMS messages

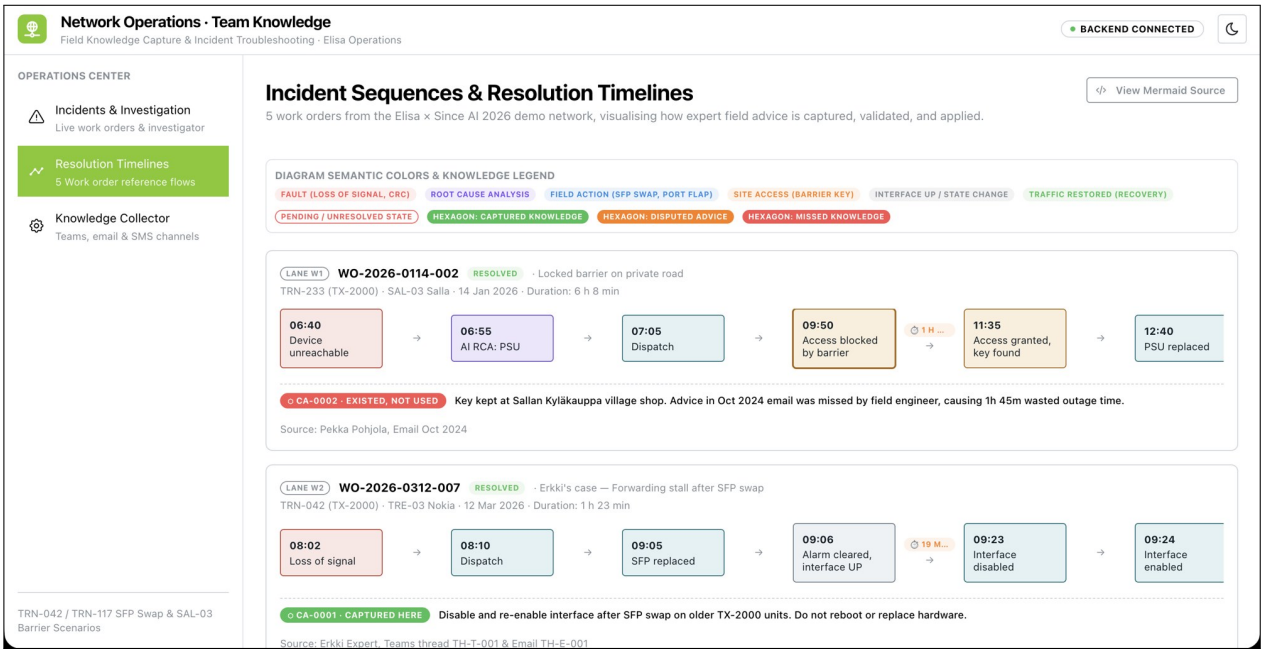


Figure 15: Resolution Timelines: each lane is one work order. Coloured steps show fault, analysis, field action, site access and recovery. Badges below a lane show the linked knowledge: `CA-0002` existed but was not used (1 h 45 min lost), `CA-0001` was captured in Erkki's case.

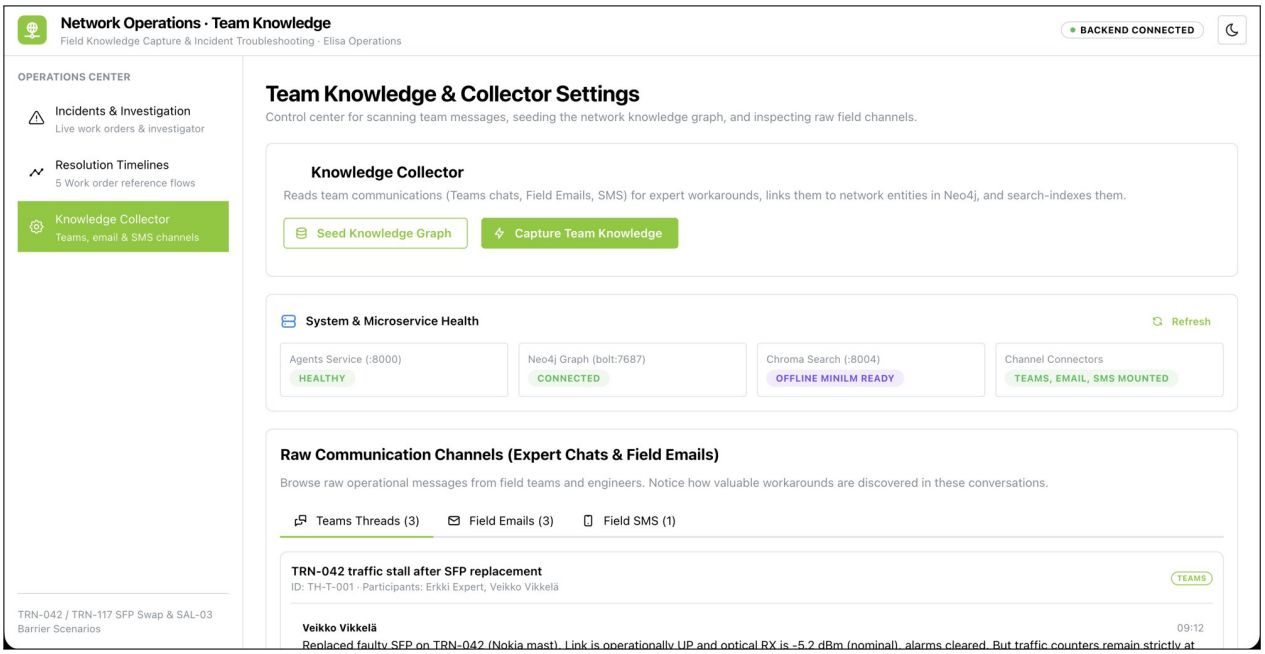


Figure 16: Knowledge Collector view: seed the graph, capture team knowledge, check health of all services, and read the raw channel messages.

9 Installation and running

9.1 Prerequisites

Table 14: What you need

Requirement	Version	Notes
Docker Engine / Docker Desktop	recent	Windows, macOS or Linux
Docker Compose	v2.20+	Included in Docker Desktop
Git	any	To clone the repository
Free RAM	about 4 GB	Neo4j, Chroma and 8 small Python/nginx containers
Free ports	3000, 8000, 7474, 7687	Change in `docker-compose.yml` if they are taken
LLM API key	optional	Without a key, the offline rule-based extractor is used
Web browser	modern	Chrome, Firefox, Edge or Safari

9.2 Installation flow

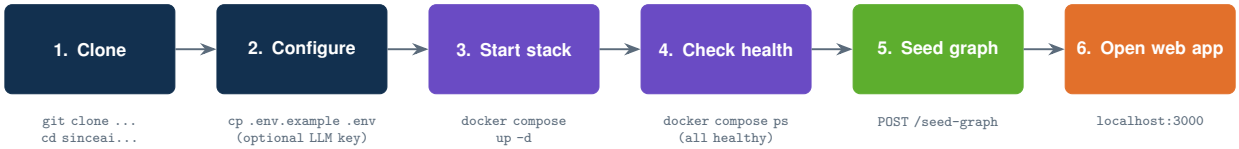


Figure 17: Six steps from zero to a running demo.

9.3 Step by step

Step 1 – Clone the repository

```
git clone https://github.com/dipankar/sinceaihackathon2026.git
cd sinceaihackathon2026
```

Step 2 – Create the environment file

```
cp .env.example .env
# Optional: put your key in .env to use a live LLM for extraction
# LLM_API_KEY=sk-...
# LLM_MODEL=gpt-4o-mini
```

Step 3 – Build and start all 10 containers

```
docker compose up -d --build
```

The first build downloads images and bakes the MiniLM embedding model into the `rag-mcp` image, so it takes a few minutes. Later starts take seconds.

Step 4 – Check that every service is healthy

```
docker compose ps # all services should show (healthy)
curl -s http://localhost:8000/health # Neo4j + MCP endpoint status
```

Step 5 – Seed the network topology and history

```
curl -s -X POST http://localhost:8000/seed-graph
# or: ./scripts/seed_graph.sh
```

This loads regions, sites, elements, ports, SFPs, services, people, five work orders, 104 events and the seeded assertions. The seed uses `MERGE`, so running it again is safe.

Step 6 – Open the web app and tools

- Web app: `http://localhost:3000`
- API documentation (Swagger): `http://localhost:8000/docs`
- Neo4j Browser: `http://localhost:7474` (user `neo4j`, password from `.env`)

9.4 Automated acceptance demo

```
./scripts/demo.sh # against the running Docker stack
./scripts/demo.sh --local # without Docker, using uv
```

The script runs: health check → baseline report (`known_solution: null`) → capture → report again (workaround returned with its source) → SAL-03 access check (keybox advice supersedes the village-shop key).

9.5 Reset to a clean demo

```
docker compose down -v # removes containers AND the neo4j/chroma volumes
docker compose up -d
curl -s -X POST http://localhost:8000/seed-graph
```

9.6 Running tests without Docker

```
cd services/agents
uv sync
uv run python -m unittest discover -v
```

9.7 Troubleshooting

Table 15: Common problems

Symptom	Fix
`agents` stays *starting*	One dependency is not healthy yet. Run `docker compose ps` and `docker compose logs <service>`.
Port 3000/8000/7474 already in use	Stop the other program or change the host port mapping in `docker-compose.yml`.
UI shows “backend not connected”	Check `curl localhost:8000/health`; wait until `agents` is healthy.
Report still shows 40 % after capture	The capture already ran once (idempotent). Reset with `docker compose down -v` and seed again.
LLM errors in the logs	Check `LLM_API_KEY` / `LLM_BASE_URL`; or leave the key empty to use the offline extractor.
Neo4j login fails	The password in `.env` must match `NEO4J_AUTH` used when the volume was first created; reset the volume.

10 Demo walkthrough

The demo uses Elisa's case on a new node. Work order `WO-2026-1107-003`: TRN-117 (TX-2000, firmware 4.1) at the Kiiminki mast (OUL-05). The SFP on port 1/1/3 was replaced; the interface is UP, alarms are clear, but traffic is zero. Services `SVC-MBH-0117` and `SVC-MBH-0125` are affected.

10.1 Step 1 – Investigate: no prior fix found

The Investigator runs its four graph steps (visible in the trace) and finds no valid assertion for this case. It does **not** guess: confidence is 40 %, and it offers to capture team advice with Teams, email and SMS pre-selected.

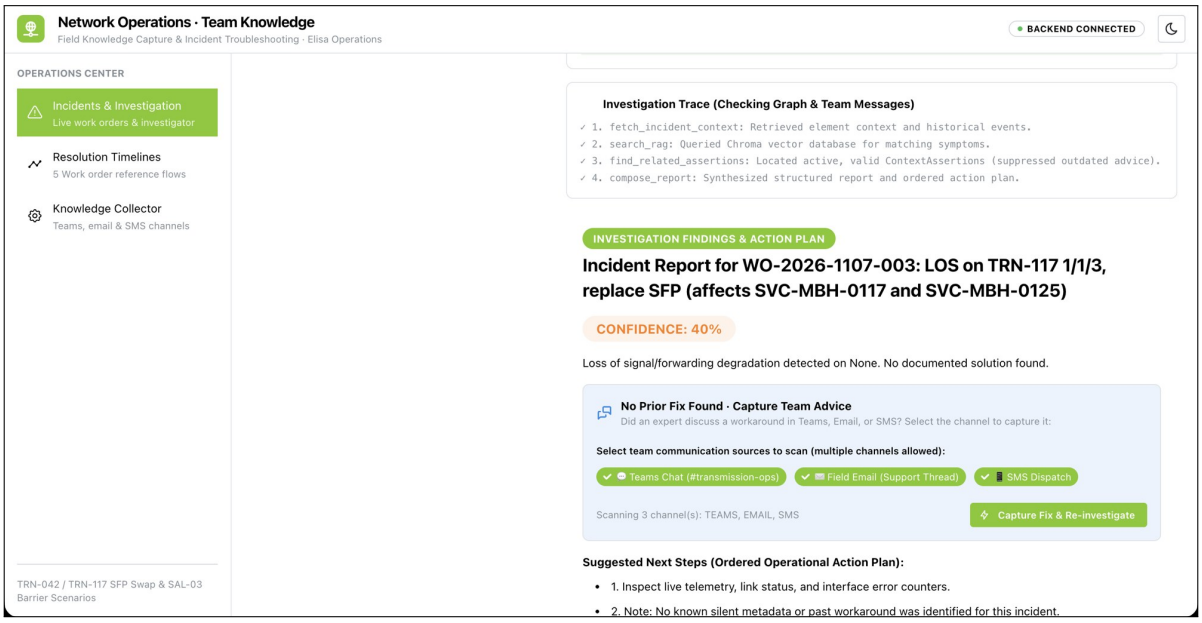


Figure 18: Before capture: investigation trace, 40 % confidence, and the "No Prior Fix Found – Capture Team Advice" card.

10.2 Step 2 – One click: capture team knowledge

The operator clicks **Capture Fix & Re-investigate**. The Collector reads the latest messages from the selected channels, extracts the expert workaround, links it to TRN-117 / TX-2000 / the work order, and adds the text to Chroma. The expert (Mikko Virtanen in this scenario) does nothing: he only wrote in Teams and email as usual.

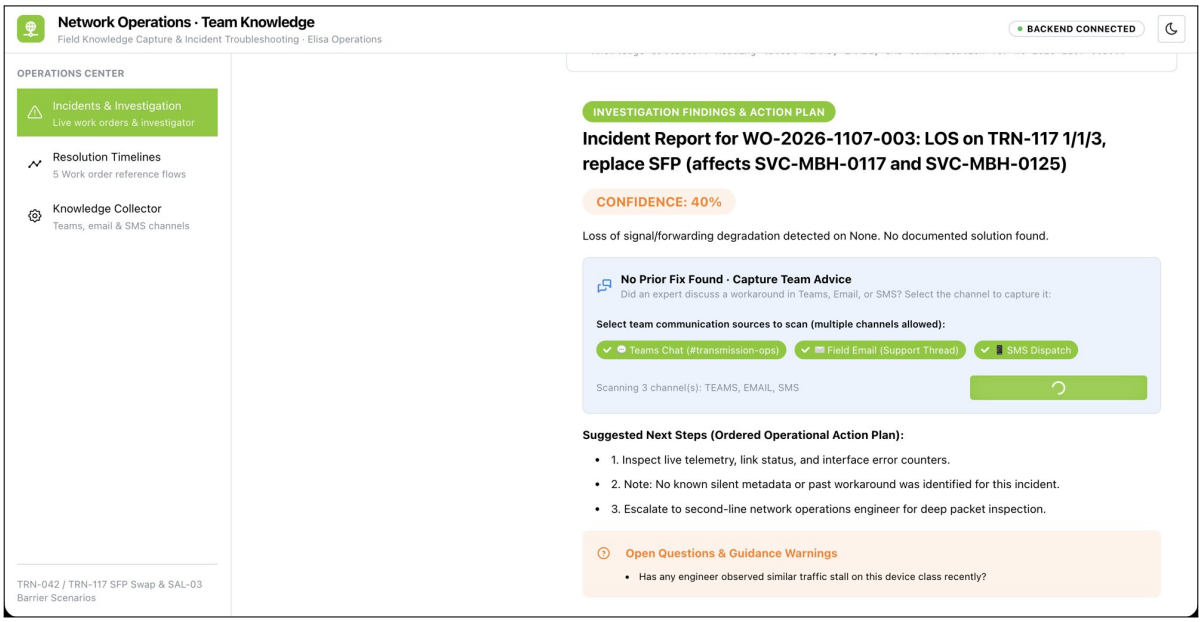


Figure 19: Capture running across three channels.

10.3 Step 3 – Investigate again: the answer with its source

The Investigator re-runs automatically. It now returns the workaround, its source (`email/TH-E-005.txt`), confidence and `valid_from`, and an ordered plan that puts the cheap, safe check first and the reboot last.

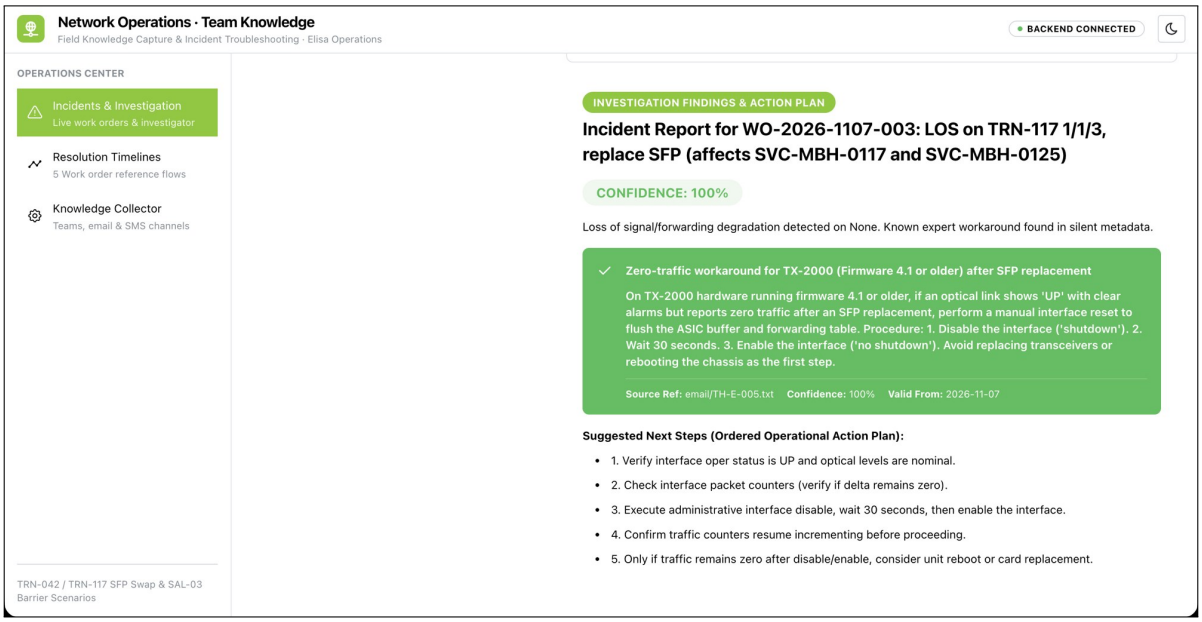


Figure 20: After capture: 100 % confidence, verified workaround card with source and validity, and a five-step plan.

This is Elisa's "strong demo" (brief, section 06)

1. The expert's knowledge is captured from chat and email **without a documentation task**.
2. It is stored as a context assertion linked to the node, the device model and the SFP-replacement context, **with source and validity**.
3. When a similar case occurs, the agent retrieves it and proposes **check counters, disable/enable the interface** before hardware is replaced or the unit is rebooted.

10.4 Step 4 – Time and conflicts

- **Access (SAL-03):** the October 2024 email says the barrier key is at the village shop (`CA-0002`). A newer email establishes a keybox with the code held by the NOC (`CA-0003`). The graph stores `CA-0003 -[:SUPERSEDES]-> CA-0002` and closes `CA-0002.valid_to`; the agent tells the engineer to call the NOC. The code itself is never stored.
- **Conflict (TRN-021, TX-3000 CRC errors):** Aino says “clean connectors first”, Mikko says “reseat the line card” (`CA-0005 -[:CONTRADICTS]-> CA-0004`). The agent shows both, caps confidence at 0.6 and recommends the least intrusive step first.
- **Version scope (TX-2000):** firmware 4.2 fixes the forwarding issue (`CA-0008`), so the bounce advice `CA-0009` applies only below 4.2 (`applies_to_sw_below = 4.2`).

11 Synthetic data: what we generated and how

Elisa asks teams to document any synthetic data. All names, node ids and messages are illustrative. No production, customer or personal data is used, and no access to Elisa systems was needed.

11.1 Principle: facts first, then words

The network, its history and the incidents were generated first (script `generate_data.py`; output: the seed file `knowledge_graph_seed.cypher`). Conversations were then written *from* those facts, so every node id, work order, timestamp and symptom in a chat matches the graph. Elisa's Appendix A Teams chat and email are used verbatim for the TRN-042 case.

11.2 The synthetic network

- 5 regions and 14 sites with real Finnish place names (Pasila, Kupittaa, Hervanta, Nokia, Kiiminki, Salla, . . .), including rural masts with special access.
- 23 network elements of fictional vendors: Nordoptic TX-2000 (legacy, 2017) and TX-3000, Arctiwave MW-500 microwave radios, Vasa Networks AG-100 routers.
- 94 ports, 88 SFPs with serials and batches, fibre links with length and validity, 6 services with their designed path.
- Firmware history as time intervals, including TX-2000 4.2 which fixes the forwarding issue.
- 104 events: alarms, AI-RCA results, dispatches, access blocked, SFP replaced, interface UP, traffic restored.

11.3 Communication channels

Table 16: Channel files and what they test

File	Topic	What it tests
`TH-T-001`	TRN-042 traffic stall (Elisa Appendix A)	Core capture; "older units" → model-level knowledge; verified outcome ("Traffic is back")
`TH-T-002`	TRN-021 CRC errors after storm	**Conflicting advice** between two experts
`TH-T-003`	MWR-261 radio ices up every winter	Seasonal knowledge, *avoided dispatch*
`TH-T-004`	TX-2000 firmware 4.2 fixes the issue	**Version-scoped** knowledge; old advice only below 4.2
`TH-T-005`	TRN-117 traffic not back (open case)	An unanswered question and a wrong guess ("bad SFP batch"): must *not* become knowledge
`TH-T-006`	TRN-117 resolution	Newest expert advice for the open incident
`TH-E-001`	TRN-042 email (Elisa Appendix A)	Same knowledge from a second channel (reinforcement)
`TH-E-002`	SAL-03 barrier, key at village shop (Oct 2024)	Access information; later **superseded**
`TH-E-003`	SAL-03 keybox, code from the NOC (Jun 2026)	**Supersedes** TH-E-002; the code is never written down

File	Topic	What it tests
TH-E-004	KEM-07 reindeer fence gate	Access and seasonal rule (call landowner in calving season)
TH-E-005	TRN-117 operational note	Firmware-scoped workaround for the open incident
TH-S-001	SMS at the SAL-03 barrier (Jan 2026)	Real cost of missing knowledge: 1 h 45 min lost

11.4 Seeded context assertions

Table 17: The 11 seeded assertions (ground truth for testing)

Id	Title	Status	Conf.	Valid from
CA-0001	TX-2000: forwarding state stuck after SFP replacement	superseded	0.80	2026-03-12
CA-0009	TX-2000 below firmware 4.2: forwarding state stuck after SFP replacement	active	0.90	2026-09-22
CA-0008	TX-2000 firmware 4.2 fixes forwarding re-init after optics swap	active	0.85	2026-09-22
CA-0002	SAL-03: locked barrier, key at Sallan Kyläkauppa	superseded	0.90	2024-10-22
CA-0003	SAL-03: barrier keybox, code from NOC	active	0.95	2026-06-02
CA-0010	SAL-03: winter road ploughed only to barrier	active	0.80	2024-10-22
CA-0007	KEM-07: close reindeer fence gate	active	0.90	2025-05-12
CA-0011	KEM-07: call landowner during calving season	active	0.85	2025-05-12
CA-0004	TX-3000 CRC errors after thunderstorm: clean connectors first	disputed	0.60	2026-07-18
CA-0005	TX-3000 CRC errors after power dip: reseat line card	disputed	0.60	2026-07-18
CA-0006	MWR-261 microwave link ices up in winter	active	0.85	2026-11-03

11.5 Five reference work orders

Table 18: Work orders used in the timelines view

Work order	Element / site	Story	Knowledge
WO-2026-0114-002	TRN-233, SAL-03 Salla	Locked barrier, 1 h 45 min lost	CA-0002 existed, not used
WO-2026-0312-007	TRN-042, TRE-03 Nokia	Erkki's case, 19 min UP without traffic	CA-0001 captured
WO-2026-0718-011	TRN-021, TRE-01 Hervanta	CRC errors, two opinions	CA-0004 / CA-0005 disputed
WO-2026-1003-004	MWR-233, SAL-03 Salla	Barrier opened with keybox code	CA-0003 confirmed
WO-2026-1107-003	TRN-117, OUL-05 Kiiminki	**Open**: UP but no traffic	matched by the agent

12 Testing and validation

12.1 Test suite

Table 19: Automated tests (`services/agents` and each MCP service)

Test	What it checks
`test_trn042_capture.py`	Collector creates a `ContextAssertion` for the TRN-042 case, linked to the element and model, with source and validity
`test_agent1.py`	Investigator report structure; no known solution before capture; correct steps after capture
`test_agent2.py`	Collector pipeline nodes and idempotency (second run adds nothing)
`test_end_to_end_cycle.py`	Full cycle: investigate → capture → investigate, as in the demo
`test_sal03_access.py`	SAL-03 access: keybox advice returned, village-shop advice superseded
`test_validity_and_conflicts.py`	Expired / superseded assertions are filtered; `CONTRADICTS` is reported as a conflict
`test_client.py`	REST API: 5 work orders, incident details, health
`test_mcp_client.py`	MCP client wiring to all servers
`services/*/test_client.py`	Each MCP server answers its tools (teams, email, sms, rag, neo4j)

Run them with `uv run python -m unittest discover -v` inside `services/agents`.

12.2 Acceptance test (`scripts/demo.sh`)

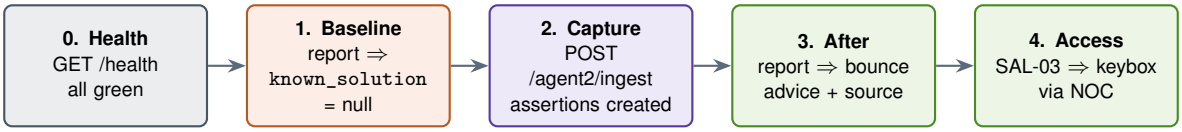


Figure 21: The acceptance sequence. Each stage fails the script if the expected result is missing.

12.3 What correctness means here

Table 20: Correctness checks and how the prototype meets them

Check	How it is enforced
No fabrication	Without a valid assertion, `known_solution` is `null` and the plan says so (Step 1 of the demo)
Right entity	Assertions are linked by id to element, model, site, access point or work order; retrieval matches on these ids
Right time	Query filters `valid_to IS NULL OR valid_to > now` and excludes targets of `SUPERSEDES`
Conflicts visible	`CONTRADICTS` or `status = disputed` ⇒ open question, both opinions, confidence ≤ 0.6
Idempotent capture	Processed-items store, deterministic ids and `MERGE` ⇒ re-runs create nothing new
Read-only analysis	Investigator only has `neo4j-mcp-ro`, which rejects write keywords
No secrets stored	Access assertions record *who* holds a code (“NOC”), never the code

13 How buildR meets Elisa's evaluation criteria

Table 21: Mapping to the criteria in the challenge brief (section 06)

Criterion	How buildR addresses it
Usability (primary)	Knowledge is taken from Teams, email and SMS, the channels experts already use. Experts fill in no form and start no documentation task. Field staff get the answer inside the incident view, with one click to capture new advice.
Relevance	Only reusable operational knowledge is stored (troubleshooting, access, known defects); chit-chat and wrong guesses are dropped (`is_operational_knowledge = false`). Each assertion has conditions, recommended action and what *not* to do.
Correct linking (entity + time)	Bound to graph entities by id (`ABOUT`, `OBSERVED_ON/IN`); `valid_from` / `valid_to` on every assertion and on time-varying relationships; supersession and conflicts are explicit edges.
Technical feasibility	Runs today with one command (10 containers, health-checked). Standard parts: Neo4j, Chroma, LangGraph, MCP, FastAPI. Works offline; any OpenAI-compatible LLM can be plugged in.
Agentic suitability	Every capability is an MCP tool, so any agent (including Elisa's own RCA agents) can call it. The Investigator returns structured JSON (plan, source, validity, confidence) that another agent can act on; read/write separation keeps analysis safe.

In one sentence

buildR turns what experts already say in Teams and email into time-aware, source-linked knowledge in the graph, so the next autonomous cycle can continue without a human stop.

14 Limitations and roadmap

14.1 Current limitations (honest status of the prototype)

Prototype scope

- **Simulated channels.** Teams, email and SMS are local files served by MCP servers with the same tool interface a real connector would have. No real Microsoft 365 or SMS gateway is connected.
- **Demo-tuned rules.** For a reliable offline demo, the fallback extractor, incident-to-message matching and some supersession and conflict checks contain rules written for the demo scenarios. With an LLM key, extraction is done by the model; entity resolution against the full graph catalogue is the next step.
- **Single tenant, no authentication.** The stack is meant for a local demo; credentials are in `.env`.
- **No human confirmation loop yet.** Assertions are stored directly; there is no "Saved as a tip ✓/ edit" reply in Teams yet.
- **Telemetry is a pointer.** `MetricSeries` nodes point to telemetry; live counters are not read by the agent yet.

14.2 Roadmap

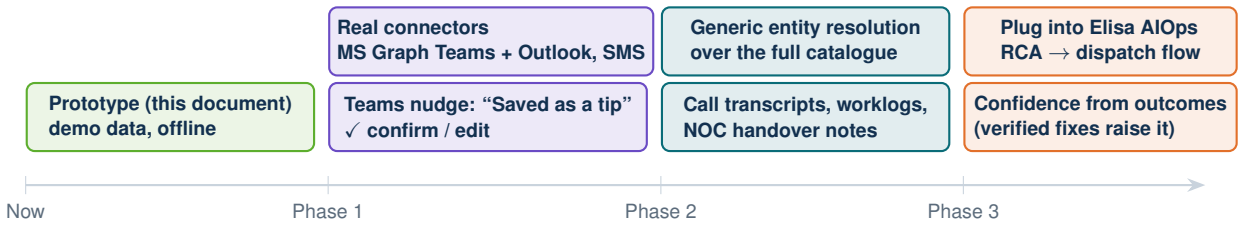


Figure 22: Next steps towards production.

14.3 Security and privacy

- Only the author's display name is stored, no other personal data.
- Secrets such as gate or keybox codes are never stored; the assertion records who holds them.
- Read and write access to the graph are separate services; analysis agents cannot modify data.
- For production: authentication on the API, secrets in a vault, role-based access to assertions, audit of who confirmed what.

A Repository structure

```
.
|-- AGENTS.md challenge specification and build rules
|-- README.md overview, architecture, quick start
|-- docker-compose.yml 10 services, health checks, ops_net
|-- .env.example configuration template
|-- data/
| |-- seed/knowledge_graph_seed.cypher graph seed (MERGE, idempotent)
| |-- teams/TH-T-001..006.json Teams threads
| |-- email/TH-E-001..005.txt emails
| `-- sms/TH-S-001.json SMS thread
|-- diagrams/incident-sequences(.mmd|-mermaid.md)
|-- docs/ PLAN.md PROGRESS.md ASSUMPTIONS.md screenshots/ slides/
|-- frontend/ React + Vite + Mantine, nginx Dockerfile
| `-- src/components/ IncidentsView.tsx SequencesFlowView.tsx SettingsView.tsx
|-- scripts/ seed_graph.sh demo.sh
`-- services/
    |-- agents/ FastAPI + LangGraph (agent1/, agent2/, main.py, models.py)
    |-- mcp-neo4j/ FastMCP Cypher adapter (READ_ONLY flag)
    |-- mcp-teams/ mcp-email/ mcp-sms/ FastMCP channel servers
    `-- rag/ FastMCP wrapper for Chroma
```

B Example source data

Teams message (`data/teams/TH-T-006.json`, excerpt):

```
{ "channel": "teams", "thread_id": "TH-T-006",
  "topic": "TRN-117 resolution - SFP traffic recovered",
  "messages": [
    { "seq": 2, "timestamp": "2026-11-07T08:50:00+02:00",
      "author": "Mikko Virtanen", "role": "Principal Transmission Specialist",
      "text": "Workaround: Disable port 1/1/3 ('interface 1/1/3 shutdown'), wait 30 seconds,
               then re-enable it ('no shutdown'). Forwarding ASIC reinitializes ..." },
    { "seq": 3, "author": "Jari Järvinen", "role": "Field Engineer",
      "text": "Flapped 1/1/3 as advised. Traffic counters jumped to 4.2 Gbps right away ..." } ] }
```

Email (`data/email/TH-E-003.txt`):

```
From: Sanna Saarinen
Date: 2026-06-02T13:05:00+03:00
Subject: SAL-03 barrier key - new arrangement

The landowner has installed a keybox at the Hirvasvaarantie barrier (SAL-03).
The key is no longer kept at Sallan Kyläkauppa.
The keybox code is available from the NOC on request - we don't send it by email.
```

Resulting assertion (as stored by the Collector):

```
(:ContextAssertion {
   id: "CA-0003", kind: "access", title: "SAL-03: barrier keybox, code from NOC",
   recommendation: "Call the NOC for the keybox code before leaving for the site",
   confidence: 0.95, status: "active",
   valid_from: 2026-06-02T13:05:00+03:00, valid_to: null,
   source_refs: ["email/TH-E-003.txt"] })
  -[:ABOUT]-> (:AccessPoint {id: "AP-SAL-03-BARRIER"})
  -[:ABOUT]-> (:Site {id: "SAL-03"})
  -[:SUPERSEDES]->(:ContextAssertion {id: "CA-0002", valid_to: 2026-06-02T13:05:00+03:00})
```

C Glossary

Term	Meaning
Silent metadata	Operational knowledge that exists only in conversations and memory, not in a system
Context assertion	One piece of captured knowledge, bound to graph entities, with validity and source
SFP	Small form-factor pluggable optical module (transceiver)
Interface bounce	Administratively disable and re-enable an interface (`shutdown` / `no shutdown`)
MCP	Model Context Protocol: a standard way for AI agents to call tools and data sources
RAG	Retrieval-augmented generation: search relevant text and give it to the AI
LangGraph	Library for building agents as explicit state graphs
NOC	Network Operations Centre
RCA	Root-cause analysis
Supersede	Newer knowledge replaces older knowledge; the old one gets a `valid_to`